EJERCICIO 5

Sistema de registro y verificación de certificados Académicos

Pablo Correa Pastor

Iago Otero Goce

1. Análisis y definición del escenario

1.1. Problema

La comprobación de certificados académicos puede depender de consultas a bases de datos administradas por la entidad emisora. Esto dificulta la verificación independiente por terceros y exige confiar en la disponibilidad y conservación del registro central.

1.2. Propuesta

El sistema propone publicar en un contrato inteligente la huella criptográfica (hash) de cada certificado y su estado. Una entidad emisora autorizada puede registrar o revocar documentos, cualquier persona puede consultar si un hash concreto figura como válido.

1.3. Justificación del uso de blockchain

- El registro es verificable por terceros usando las mismas reglas del contrato.
- Los cambios de estado se registran como transacciones, los eventos facilitan seguir los registros y revocaciones.
- La verificación no requiere divulgar el contenido del certificado en el contrato.

El prototipo se ha probado en Remix VM. Para conseguir la verificación pública entre instituciones habría que desplegarlo en una red pública y asociar la dirección emisora a una institución real.

2. Requisitos del sistema

Código	Requisito
R1	Registrar hashes de certificados no registrados anteriormente.
R2	Permitir únicamente al emisor registrar y revocar certificados.
R3	Consultar públicamente si un certificado existe y no está revocado.
R4	Revocar certificados existentes sin borrar su historial.
R5	Consultar el estado y la fecha de registro de un certificado.
R6	Emitir eventos cuando se registre o revoque un certificado.

3. Diseño

3.1. Actores y casos de uso

Se han identificado dos actores: el emisor (cuenta que despliega el contrato) y el verificador (cualquier cuenta que consulte la información). La Figura 1 resume las interacciones principales.



Figura 1. Diagrama de casos de uso del contrato inteligente.

3.2. Datos, funciones y reglas

Elemento	Tipo / función	Propósito
emisor	address immutable	Cuenta autorizada para modificar el registro.
Certificado	struct	Campos existe, revocado y fechaRegistro.
certificados	mapping(bytes32 => Certificado)	Asocia el hash con su estado.
registrarCertificado	Función restringida	Registra un hash válido no duplicado.
verificarCertificado	Función de lectura	Devuelve true sólo si existe y no está revocado.
revocarCertificado	Función restringida	Marca un certificado existente como revocado.
consultarCertificado	Función de lectura	Devuelve existencia, revocación y fecha.
calcularHash	Función	Calcula keccak256 del texto usado en la demostración.

4. Implementación

Se ha creado el archivo CertificadosAcademicos.sol con Solidity 0.8.30. El constructor asigna a la cuenta desplegadora el papel de emisor. La restricción de acceso se implementa mediante el modificador soloEmisor, que utiliza require(msg.sender == emisor, "No autorizado").

El contrato utiliza un identificador bytes32 por certificado, una estructura para almacenar existencia, revocación y fecha de registro, y un mapping para localizar cada entrada. Los eventos CertificadoRegistrado y CertificadoRevocado permiten identificar las modificaciones en el registro de transacciones.

4.1. Optimización del gas

- La dirección del emisor se declara immutable y se fija una sola vez en el constructor.
- La fecha se guarda como uint64, suficiente para almacenar la marca de tiempo utilizada.
- Los parámetros de solo lectura de tipo string emplean calldata para evitar copias innecesarias.
- Las consultas view y pure pueden realizarse localmente sin enviar una transacción de cambio de estado.

5. Pruebas funcionales en Remix IDE

Entorno: Remix VM (Osaka), cuenta 1 como emisor y cuenta 2 como usuario no autorizado. Se utilizó el texto de prueba “Certificado alumno 001” para obtener un hash, registrarlo y verificarlo. Después se revocó el mismo certificado. Para el control de acceso se utilizó un segundo hash generado a partir de “Certificado alumno 002”.

Prueba	Resultado esperado	Resultado observado
P1. Registro	El emisor registra un hash nuevo.	Transacción correcta.
P2. Verificación	Un certificado activo devuelve true.	bool: true.
P3. Revocación	Tras revocar, el certificado devuelve false.	Transacción correcta, bool: false.
P4. Acceso	Una cuenta distinta del emisor no puede registrar.	Transacción revertida: “No autorizado”.

5.1. Obtención del hash y registro

Se invocó calcularHash con el texto de prueba y se empleó el valor bytes32 generado para llamar a registrarCertificado desde Account 1.

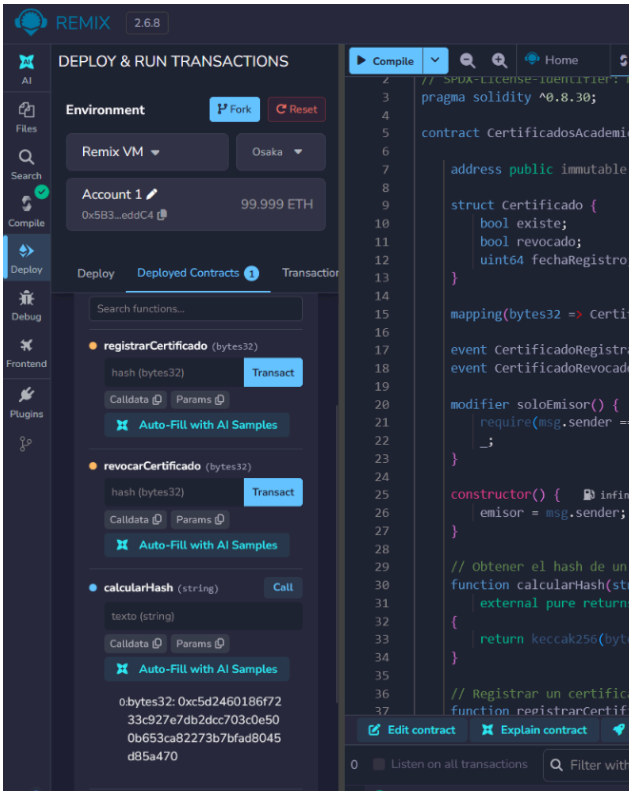


Figura 2. Identificador criptográfico de un certificado de prueba obtenido en Remix.

5.2. Verificación correcta

Tras registrar el hash, verificarCertificado devolvió true. La consola también muestra la transacción de registro completada.

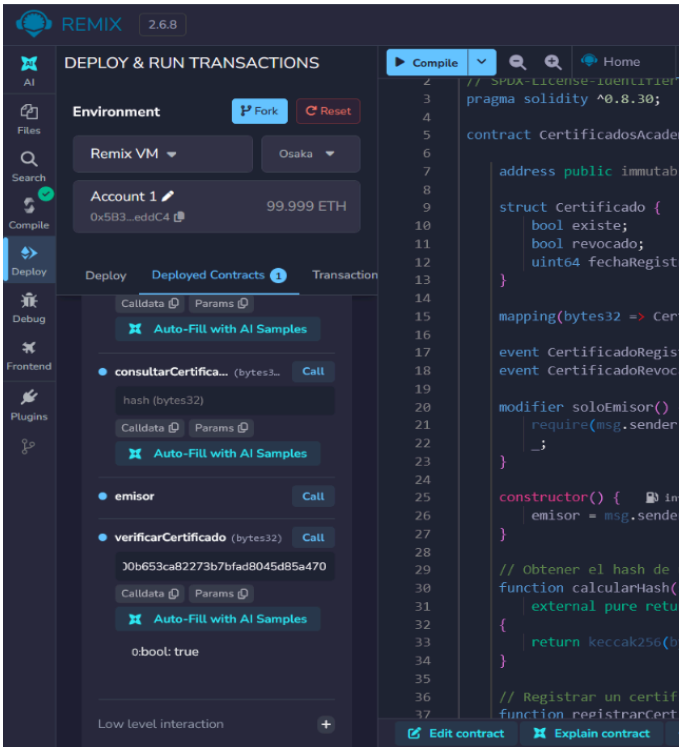


Figura 3. Confirmación del registro y consulta de validez positiva.

5.3. Revocación

Desde Account 1 se llamó a revocarCertificado. La consulta de validez sobre el hash devolvió false, ya que el certificado sigue registrado pero ha pasado al estado revocado.

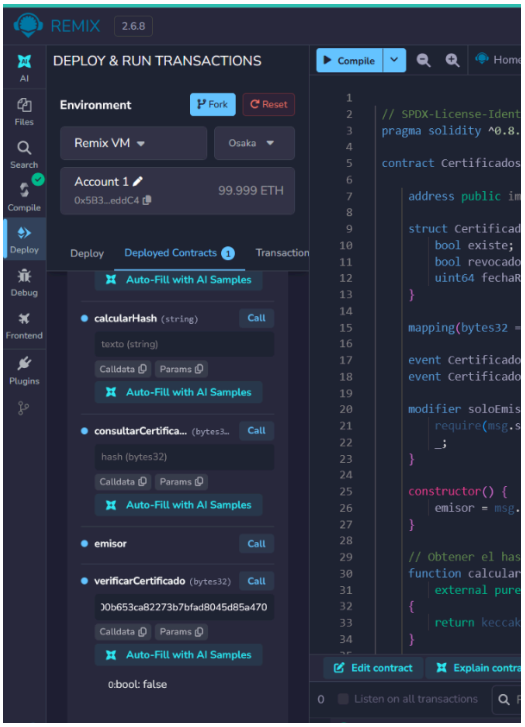


Figura 4. Revocación correcta, la verificación posterior devuelve false.

5.4. Control de acceso

Desde Account 2 se intentó registrar un segundo certificado. La ejecución fue revertida con el mensaje “No autorizado”, confirmando el funcionamiento del modificador soloEmisor.

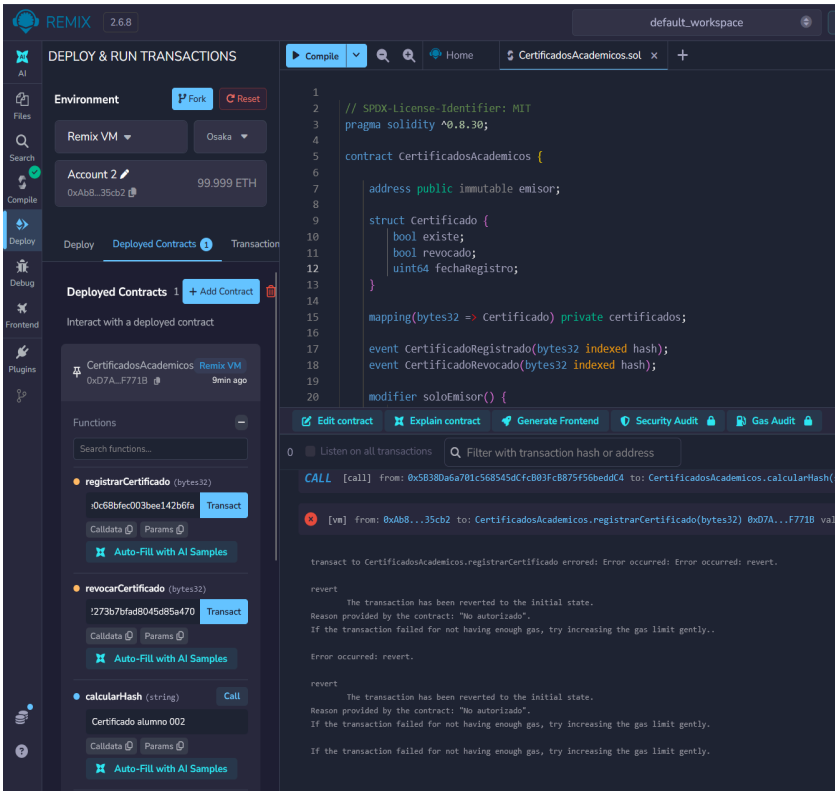


Figura 5. Rechazo de registro desde Account 2 y mensaje de error.

6. Consideraciones de seguridad y limitaciones

- Solo la dirección asignada como emisor puede registrar o revocar certificados, pero el contrato no verifica por sí solo la identidad jurídica de dicha institución.
- El hash demuestra coincidencia con los datos utilizados para calcularlo, no acredita por sí mismo la autenticidad del contenido. Para un uso real se necesitaría gestionar el documento original y el reconocimiento del emisor.
- Las pruebas usan hashes de textos de demostración, no archivos académicos reales. Un sistema de producción calcularía el hash del documento fuera de la blockchain.
- El estado de revocación permanece consultable, no se borra el registro histórico.
- No se han implementado múltiples emisores, caducidades, almacenamiento externo de documentos ni una interfaz gráfica para usuarios finales.

7. Conclusiones

Se ha diseñado e implementado un prototipo funcional de verificación de certificados académicos mediante un smart contract. Las evidencias obtenidas en Remix muestran que el emisor registra un certificado, cualquier usuario puede consultar su validez, el emisor puede revocarlo y una cuenta diferente no puede modificar el registro. Estas funciones cubren el caso de uso planteado y permiten aplicar estructuras, mappings, modificadores, eventos y hashes en Solidity.

Anexo A. Código completo del contrato

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;

contract CertificadosAcademicos {
    address public immutable emisor;

    struct Certificado {
        bool existe;
        bool revocado;
        uint64 fechaRegistro;
    }

    mapping(bytes32 => Certificado) private certificados;

    event CertificadoRegistrado(bytes32 indexed hash);
    event CertificadoRevocado(bytes32 indexed hash);

    modifier soloEmisor() {
        require(msg.sender == emisor, "No autorizado");
        _;
    }

    constructor() {
        emisor = msg.sender;
    }

    function calcularHash(string calldata texto)
        external pure returns (bytes32)
    {
        return keccak256(bytes(texto));
    }

    function registrarCertificado(bytes32 hash)
        external soloEmisor
    {
        require(hash != bytes32(0), "Hash invalido");
        require(!certificados[hash].existe, "Ya registrado");

        certificados[hash] = Certificado(
            true,
            false,
            uint64(block.timestamp)
        );
        emit CertificadoRegistrado(hash);
    }

    function verificarCertificado(bytes32 hash)
        external view returns (bool)
    {
        return certificados[hash].existe &&
               !certificados[hash].revocado;
    }

    function revocarCertificado(bytes32 hash)
        external soloEmisor
    {
        require(certificados[hash].existe, "No existe");
        require(!certificados[hash].revocado, "Ya revocado");

        certificados[hash].revocado = true;
        emit CertificadoRevocado(hash);
    }

    function consultarCertificado(bytes32 hash)
        external view returns (bool, bool, uint64)
    {
        Certificado storage c = certificados[hash];
        return (c.existe, c.revocado, c.fechaRegistro);
    }
}
```